Performance Report: Autonomous Vehicle Vision System

Jayashree Janani Purushothaman
Master of Computer Science
University of New Brunswick
Student #: 3772545
jayashree.purushothaman@unb.ca

Ruturaj Sushil Ghodke
Master of Computer Science
University of New Brunswick
Student #: 3775789
ruturaj.ghodke@unb.ca

Thirunavukarasu Muthu Valliappan
Master of Computer Science
University of New Brunswick
Student #: 3785346
mv.t@unb.ca

I. Introduction

Real-time autonomous vehicle (AV) vision systems require near-instantaneous processing of high-resolution sensor data to ensure safety-critical decision-making. Central to these systems are sequential preprocessing pipelines, specifically Grayscale conversion, Gaussian blurring, and Sobel edge detection, which reduce noise and extract essential structural features like road boundaries and obstacles. However, the computational intensity of these stencil-based operations necessitates the use of high-performance parallel architectures.

This study evaluates the scalability and performance trade-offs of such a pipeline across three distinct programming models: a serial baseline, a shared-memory implementation using OpenMP, and a distributed-memory model via MPI. By analyzing runtime behavior, speedup, and efficiency across varying workload sizes, we identify the primary bottlenecks inherent in data locality and inter-process communication. The following report details the implementation strategies and provides a comparative analysis of how these parallel models handle the transition from localized computation to large-scale distributed processing.

II. Problem Definition and Inputs

A. Pipeline Overview

The computational core of this study involves a multi-stage vision pipeline designed to transform raw sensor data into actionable structural information. To simulate the early-stage processing requirements of an autonomous vehicle, we implement a sequential dependency chain consisting of three distinct image kernels:

1) **Grayscale Conversion**: An $O(N)$ pointwise transformation that maps three-channel RGB data to a single luminance channel, significantly reducing the memory footprint for subsequent stages.
2) **Gaussian Blur**: A spatial convolution used to suppress high-frequency noise. This stage introduces local data dependencies, as each output pixel requires a neighborhood of input pixels.
3) **Sobel Edge Detection**: A directional derivative operator used to highlight structural boundaries. Like the Gaussian stage, this requires boundary-aware computation (halos) to determine edge intensity.

The output of each stage becomes the input of the next stage, forming the following pipeline:

$$\text{Input Image} \rightarrow \text{Grayscale} \rightarrow \text{Gaussian Blur} \rightarrow \text{Sobel Edge Detection} \rightarrow \text{Output Image}$$

B. Dataset and Workloads

We used a publicly available vehicle dashcam dataset from Kaggle [1]. To evaluate scalability, we tested the application on three workload sizes:

TABLE I: Input workload sizes used in the experiments

Workload	Resolution	Purpose
Small	640 × 480	Debugging, rapid correctness testing
Medium	1920 × 1080	Standard full-HD dashcam workload
Large	4K	Stress-testing computation and MPI communication overhead

The program outputs a processed image file representing the final edge-detected result.

III. Experimental Setup

All experiments were conducted using three implementations of the image-processing pipeline: a serial baseline, a shared-memory OpenMP version, and a distributed-memory MPI version. Each implementation executed the same three-stage processing workflow consisting of grayscale conversion, Gaussian blur filtering, and Sobel edge detection.

Performance evaluation was performed using three datasets categorized as small (640×480), medium (1920×1080), and large (4K resolution). Each dataset contained multiple images processed sequentially within batch execution, and the reported runtime represents the total processing time required to process all images in each dataset.

To ensure reliable benchmarking, each experiment was repeated multiple times for every dataset and parallel configuration. The first run was discarded to minimize initialization overhead such as cache warm-up and file system latency, and the remaining runs were averaged to obtain the final execution time.

The OpenMP implementation was evaluated using 1, 2, 4, and 8 threads to analyze shared-memory scalability. The MPI implementation was evaluated using 1, 2, 4, and 8 processes to study distributed-memory performance and communication overhead. Speedup and efficiency were computed relative to the serial baseline execution time.

All experiments were conducted on a system equipped with an Intel Core Ultra 7 processor with 16 cores/threads and 16 GB of RAM, running Windows 11 with WSL (Ubuntu). The implementations were compiled using GCC (MinGW) version 15.2.0, and MPI experiments were executed using Open MPI version 4.1.6.

TABLE II: Experimental environment

Component	Specification
Processor	Intel core ultra 7
Cores / Threads	16
RAM	16 GB
Operating System	Windows 11 with WSL(Ubuntu)
Compiler	GCC (MinGW) 15.2.0
MPI Library	mpirun (Open MPI) 4.1.6

The implementation uses the public-domain single-header libraries `stb_image.h` and `stb_image_write.h` for image input and output operations[2].

IV. Design Strategy

The system was designed as a modular image-processing pipeline that applies three sequential pre-processing stages: grayscale conversion, Gaussian blur filtering, and Sobel edge detection. These stages were selected because they represent common operations used in early computer vision pipelines and provide a suitable structure for evaluating different parallelization strategies under both shared-memory and distributed-memory models.

The implementation consists of three program variants: a serial baseline version, an OpenMP shared-memory version, and an MPI distributed-memory version. Each version processes images from a dataset directory and applies the same pipeline stages to ensure correctness and comparability across implementations. The output of each stage is stored in intermediate buffers, allowing the pipeline to be executed sequentially while maintaining clear data dependencies between stages.

A. Serial Design Strategy

The serial implementation processes images sequentially from a specified input directory. Each image is loaded into memory using the `stb_image` library and passed through the three-stage filtering pipeline consisting of grayscale conversion, Gaussian blur, and Sobel edge detection. Intermediate buffers are allocated for each stage and released after processing to maintain controlled memory usage.

Execution time is measured both per image and for the entire dataset. The total runtime recorded across all images serves as the baseline reference for evaluating speedup and efficiency in the OpenMP and MPI implementations. This sequential batch-processing structure ensures correctness and provides a consistent performance benchmark.

B. OpenMP Design Strategy

The OpenMP implementation parallelizes the computationally intensive filtering stages using loop-level parallelism. Since the grayscale, blur, and Sobel filters operate independently on individual pixels or small neighborhoods, they are well suited for shared-memory parallel execution.

Parallelization is implemented inside the filter functions using `#pragma omp parallel for` directives with static scheduling. Each thread processes a subset of pixels, enabling efficient workload distribution while minimizing scheduling overhead. The pipeline structure remains unchanged from the serial version, ensuring identical output behavior while improving execution speed through concurrent processing.

This approach allows scalability to be evaluated across different thread counts while preserving the simplicity of the sequential pipeline structure.

C. MPI Design Strategy

The MPI implementation distributes image-processing work across multiple processes using domain decomposition along image rows. In this approach, the root process reads each input image and broadcasts its dimensions to all participating processes. The image data is then partitioned into contiguous row blocks and distributed using `MPI_Scatterv`.

Since Gaussian blur and Sobel edge detection require neighboring pixel information, halo exchange is performed between adjacent processes to maintain correctness at partition boundaries. Two halo exchange stages are used: one before applying the blur filter and another before applying the Sobel filter. These exchanges ensure that each process has access to the required boundary pixels for convolution operations.

After processing, the filtered image segments are collected at the root process using `MPI_Gatherv`, where the final output image is reconstructed and written to disk. This distributed-memory strategy enables evaluation of communication overhead and scalability effects associated with stencil-based image processing.

D. Dataset Preparation Strategy

To support consistent benchmarking across workload sizes, images were resized into three resolution categories: small (640×480), medium (1920×1080), and large ($4K$). A preprocessing utility was implemented using the `stb_image_resize` library to standardize dataset dimensions before execution. This ensured fair comparison across serial, OpenMP, and MPI experiments by maintaining identical input characteristics.

Overall, the design separates image loading, filtering operations, and parallelization logic into modular components, allowing the same pipeline structure to be reused across execution models while enabling systematic evaluation of performance scalability.

V. Image Processing Pipeline

The proposed pipeline consists of grayscale conversion, Gaussian blur filtering, and Sobel edge detection applied sequentially to the input RGB image (Figure: 1).



Fig. 1: Original RGB input image used for the image-processing pipeline

In the first stage, the input RGB image is converted into a grayscale representation. This step reduces computational complexity by eliminating color channels while preserving essential structural information required for subsequent filtering operations. Since each pixel can be processed independently, grayscale conversion is well suited for parallel execution using both shared-memory and distributed-memory approaches.



Fig. 2: Grayscale representation of the input image after color-channel reduction

In the second stage, a Gaussian blur filter is applied to smooth the grayscale image and suppress high-frequency noise. Noise reduction improves the stability and accuracy of gradient-based edge detection in the following stage. Because both Gaussian filtering and Sobel edge detection rely on neighboring pixel values, halo regions must be exchanged between adjacent partitions in the MPI implementation to ensure correct boundary computation.



Fig. 3: Gaussian-blurred image showing noise suppression prior to edge detection

In the final stage, the Sobel operator is applied to detect edges by computing intensity gradients in both horizontal and vertical directions. The resulting edge map highlights important structural features such as lane markings, road boundaries, and object contours that are critical for autonomous vehicle perception tasks.



Fig. 4: Final edge map produced using the Sobel operator

The same three-stage pipeline is used across the serial, OpenMP, and MPI implementations to ensure consistency in correctness verification and enable fair performance comparison between execution models. Figure 2, 3 and 4 illustrates the intermediate outputs produced at each stage of the processing workflow. This staged pipeline structure enables efficient loop-level parallelization in OpenMP and structured domain decomposition with halo exchange in the MPI implementation.

VI. Results

A. Serial Baseline

The serial implementation serves as the reference baseline for evaluating the performance of the parallel OpenMP and MPI versions. All image-processing operations, including grayscale conversion, Gaussian blur filtering, and Sobel edge detection, were executed sequentially using a single processor.

Table III summarizes the execution time required to process the small, medium, and large datasets. As expected, execution time increases with image resolution due to the higher computational workload associated with larger image sizes. These baseline measurements were used to compute speedup and efficiency for the parallel implementations.

TABLE III: Serial Execution Time Across Datasets

Dataset	**Execution Time (s)**
Small (640×480)	76.825
Medium (1920×1080)	267.804
Large (4K)	718.468

B. OpenMP Performance

The OpenMP results demonstrate that performance improvement depends strongly on dataset size and processor count. Table IV summarizes the runtime, speedup, and efficiency observed across different workloads.

1) Small Dataset: For the small dataset, increasing the number of processors does not improve performance. Instead, the runtime increases from 76.825 s using one processor to 805.823 s with eight processors, while speedup decreases from 1.00 to 0.09 and efficiency drops to 0.01. This behavior indicates that parallel overhead dominates computation time for small workloads, resulting in performance degradation as processor count increases.

2) Medium Dataset: For the medium dataset, performance improves significantly when two processors are used, achieving the highest speedup of 2.79 and an efficiency value greater than one (1.39), indicating superlinear speedup. However, performance decreases again when the number of processors increases to four and eight. This suggests that although the workload benefits from moderate parallelism, excessive threads introduce synchronization and scheduling overhead that limits scalability beyond two processors.

3) Large Dataset: The large dataset demonstrates the best OpenMP scaling behavior among all workloads. A speedup of 3.44 is achieved using two processors, while performance remains acceptable at four processors with a speedup of 1.71 and still reasonable at eight processors. These results indicate that OpenMP achieves more effective parallelization for larger workloads, where computation dominates thread management overhead and improves overall execution efficiency.

TABLE IV: OpenMP Performance Results

Dataset	**Processors (P)**	**Time (s)**	**Speedup (S)**	**Efficiency (E)**
Small	1 (Serial)	76.825	1.00	1.00
	2	77.945	0.98	0.49
	4	267.242	0.28	0.07
	8	805.823	0.09	0.01
Medium	1 (Serial)	267.804	1.00	1.00
	2	96.015	2.79	1.39
	4	280.008	0.95	0.24
	8	752.268	0.35	0.04
Large	1 (Serial)	718.468	1.00	1.00
	2	208.636	3.44	1.72
	4	418.244	1.71	0.43
	8	815.955	0.88	0.11

C. MPI Performance

MPI exhibits different performance behavior compared to OpenMP because communication overhead plays a significant role in execution time. Table V summarizes the runtime, speedup, and efficiency observed across different workloads.

1) Small Dataset: For the small dataset, performance degrades significantly as the number of processors increases. The speedup drops to 0.13 and efficiency decreases to 0.01 when eight processors are used. This behavior indicates that MPI is inefficient for small workloads because communication overhead dominates computation time, resulting in reduced overall performance.

2) Medium Dataset: For the medium dataset, performance improves substantially when two processors are used, achieving a speedup of 2.70 and an efficiency value greater than one (1.35), indicating superlinear speedup. However, performance decreases as the number of processors increases further. This suggests that moderate processor counts improve performance, but excessive communication between processes limits scalability beyond two processors.

3) Large Dataset: The large dataset demonstrates the best MPI performance among all workloads. A speedup of 3.48 is achieved using two processors, while performance remains acceptable at four processors with a speedup of 1.43. However, performance drops sharply when eight processors are used. These results indicate that MPI performs well for larger datasets at moderate processor counts but suffers scalability limitations when communication overhead increases at higher processor counts.

TABLE V: MPI Performance Results

Dataset	Processors (P)	Time (s)	Speedup (S)	Efficiency (E)
Small	1 (Serial)	76.825	1.00	1.00
	2	86.507	0.88	0.44
	4	231.798	0.33	0.08
	8	578.559	0.13	0.01
Medium	1 (Serial)	267.804	1.00	1.00
	2	98.936	2.70	1.35
	4	338.401	0.79	0.20
	8	940.697	0.28	0.03
Large	1 (Serial)	718.468	1.00	1.00
	2	206.078	3.48	1.74
	4	500.580	1.43	0.36
	8	1603.103	0.44	0.05

D. Plots

Figures 5, 6, and 7 illustrate the execution time and scaling behavior of OpenMP and MPI across different dataset sizes. The results show that both programming models achieve their best performance at moderate processor counts, particularly when two processors are used. For the small dataset (640×480), neither OpenMP nor MPI benefits from parallel execution. Execution time increases as the number of processors grows, indicating that parallel overhead dominates computation. MPI performs slightly better than OpenMP at higher processor counts in this case, but overall performance remains worse than serial execution.

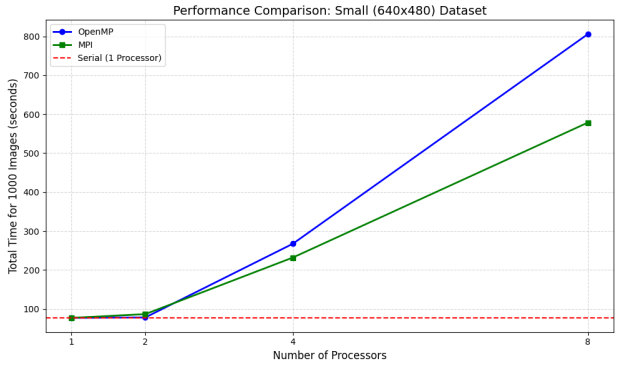


Fig. 5: Performance Comparison: Small (640 x 480) Dataset

For the medium dataset (1920×1080), both OpenMP and MPI achieve their best performance at two processors, demonstrating superlinear speedup. However, performance decreases when the processor count increases beyond this point due to synchronization overhead in OpenMP and communication overhead in MPI. OpenMP maintains slightly better scalability than MPI at higher processor counts for this dataset.



Fig. 6: Performance Comparison: Medium (1920 x 1080) Dataset

For the large dataset (4K resolution), the advantages of parallel execution become more evident. Both OpenMP and MPI achieve their highest speedup at two processors, with OpenMP maintaining more stable performance as processor counts increase. MPI performance degrades more rapidly at higher processor counts due to increased communication costs between processes.

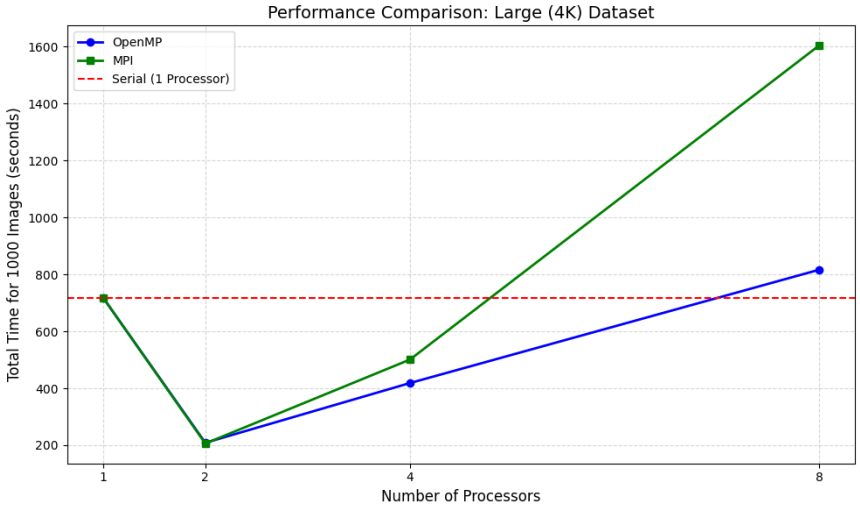


Fig. 7: Performance Comparison: Large (4K) Dataset

The speedup comparison plot further confirms that ideal linear scaling is not achieved in practice (Figure: 8). Instead, performance improves only up to an optimal processor count before overhead begins to dominate execution time. Similarly, the efficiency comparison shows superlinear behavior at two processors for medium and large datasets, which can be attributed to improved cache utilization and memory access patterns.

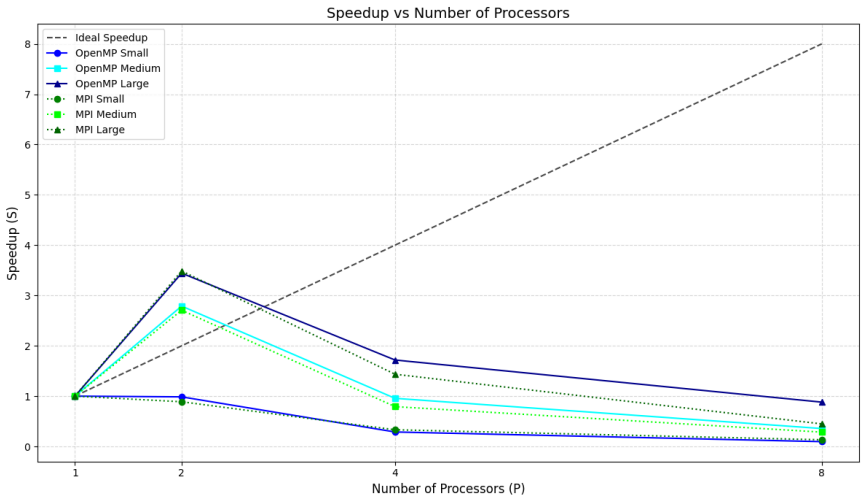


Fig. 8: Speedup Analysis

The OpenMP scaling plot across image resolutions shows that the best performance is consistently achieved when two processors are used for all dataset sizes. While execution time decreases significantly from one to two processors, performance degrades when the processor count increases further due to thread management and synchronization overhead. The improvement is most noticeable for the large (4K) dataset, indicating that OpenMP benefits more from parallelization as workload size increases. This confirms that larger computational workloads better amortize parallel overhead.

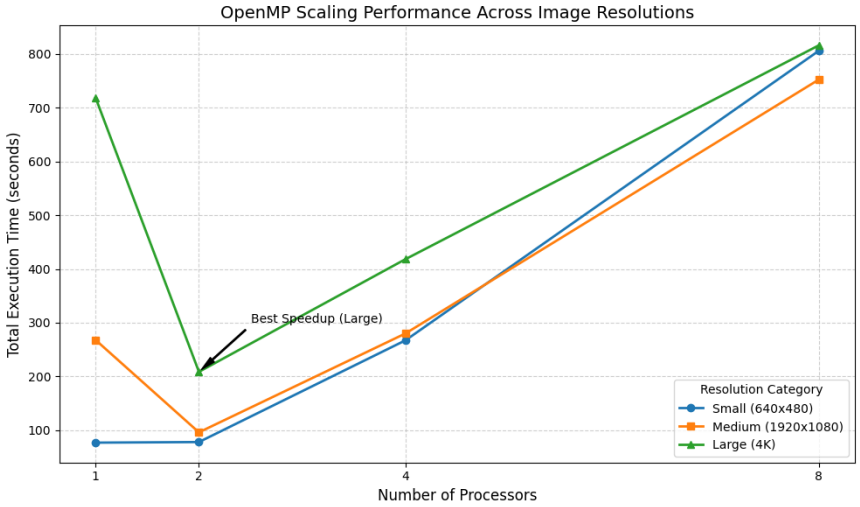


Fig. 9: OpenMP Scaling Across Image Resolutions

The MPI scaling results across image resolutions show that performance improves significantly when two processors are used, particularly for medium and large datasets. However, execution time increases noticeably as the number of processors grows beyond this point. This behavior indicates that communication overhead between processes becomes a dominant factor at higher processor counts. Similar to OpenMP, the largest performance improvement is observed for the 4K dataset, demonstrating that MPI is more effective for computationally intensive workloads but less scalable when communication costs increase.

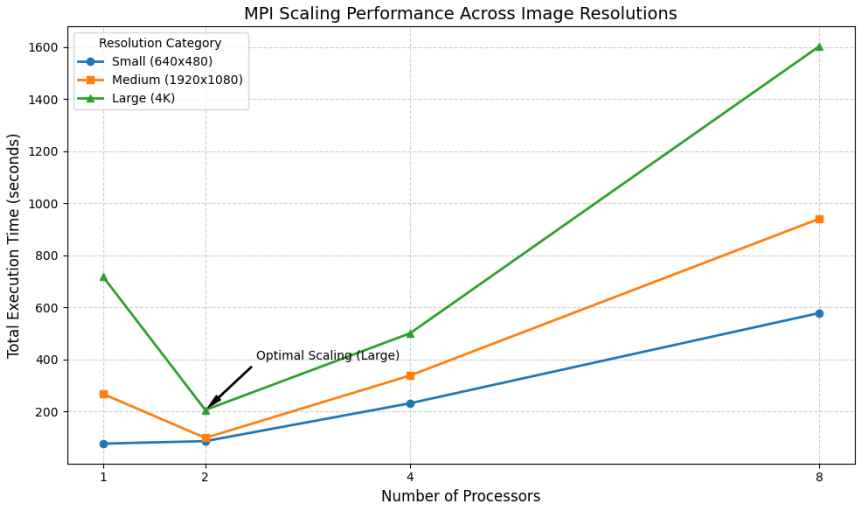


Fig. 10: MPI Scaling Across Image Resolutions

The consolidated parallel efficiency comparison between OpenMP and MPI demonstrates that the highest efficiency values are achieved when two processors are used, particularly for medium and large datasets. In several cases, efficiency exceeds one, indicating superlinear speedup caused by improved cache utilization and memory access behavior. However, efficiency decreases significantly as the number of processors increases beyond two, highlighting the impact of synchronization overhead in OpenMP and communication overhead in MPI. Overall, OpenMP maintains more stable efficiency across higher processor counts compared to MPI.

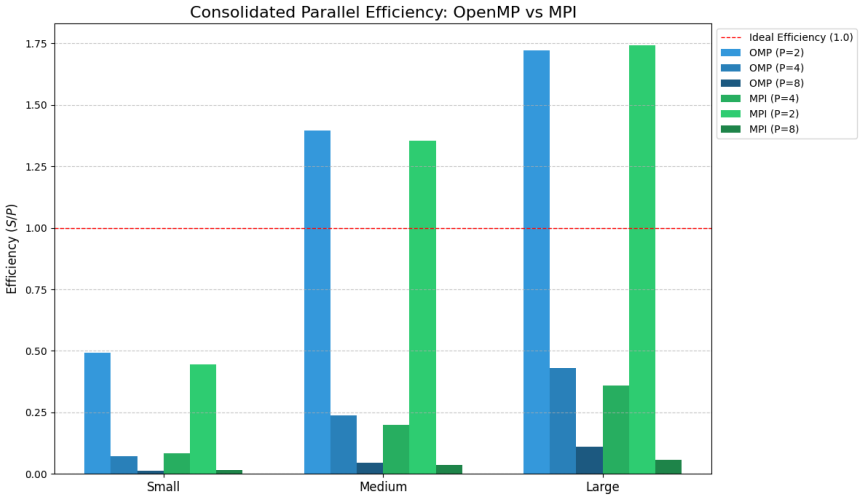


Fig. 11: Consolidated Parallel Efficiency

E. OpenMP Analysis

The OpenMP implementation is well suited for this application because the image-processing pipeline consists primarily of loop-level data-parallel operations. In the grayscale conversion stage, each pixel can be processed independently without requiring synchronization. Similarly, Gaussian blur and Sobel edge detection operate on small local neighborhoods, allowing efficient parallelization using shared-memory thread-level parallelism.

Experimental results show that OpenMP achieves its best performance at moderate thread counts, particularly when two processors are used. For medium and large image resolutions, superlinear speedup is observed, which can be attributed to improved cache utilization and memory locality. However, performance decreases as the number of threads increases further. This behavior is primarily caused by memory bandwidth limitations, cache contention, and thread management overhead, which reduce scalability at higher processor counts. As a result, OpenMP provides effective acceleration for larger workloads but does not maintain linear scaling as the number of threads increases.

F. MPI Analysis

The MPI implementation enables distributed-memory parallelism but introduces additional communication overhead compared to OpenMP. This overhead is especially significant for stencil-based filters such as Gaussian blur and Sobel edge detection, where halo exchanges are required between neighboring partitions. Consequently, MPI shows weaker performance for smaller image resolutions, where communication costs dominate useful computation.

For medium and large workloads, MPI performance improves significantly because the computation-to-communication ratio becomes more favorable. The highest speedup is observed when two processors are used, after which performance declines as the number of processes increases. This reduction in scalability is caused by process synchronization overhead, message passing latency, and final data aggregation costs. Overall, MPI demonstrates strong performance improvements for larger datasets but remains sensitive to communication overhead at higher processor counts.

G. Amdahl's Law and Bottlenecks

The achievable speedup of the application is fundamentally limited by its serial components, as described by Amdahl's Law:

$$S(N) = \frac{1}{f + \frac{1-f}{N}},$$

where f represents the serial fraction of the program and N denotes the number of parallel workers.

Although the pixel-processing stages are highly parallelizable, several components remain partially serial and restrict overall scalability. These include image input and output operations, boundary-condition handling, MPI communication and synchronization steps, and initialization overhead such as memory allocation and process setup. These factors explain why the measured speedup deviates from ideal linear scaling and why performance improvements diminish as the number of processors increases.

The scripts used to generate performance measurements and scaling plots are available in the project repository [3].

VII. Challenges and Solutions

A. MPI Halo Exchange Artifact Issue

During the development of the MPI implementation, horizontal artifact lines appeared in the output images when running with multiple processes (e.g., 2 and 4 processes as shown in Figure: 12 and Figure: 13). These artifacts occurred at the boundaries between image partitions assigned to different processes.

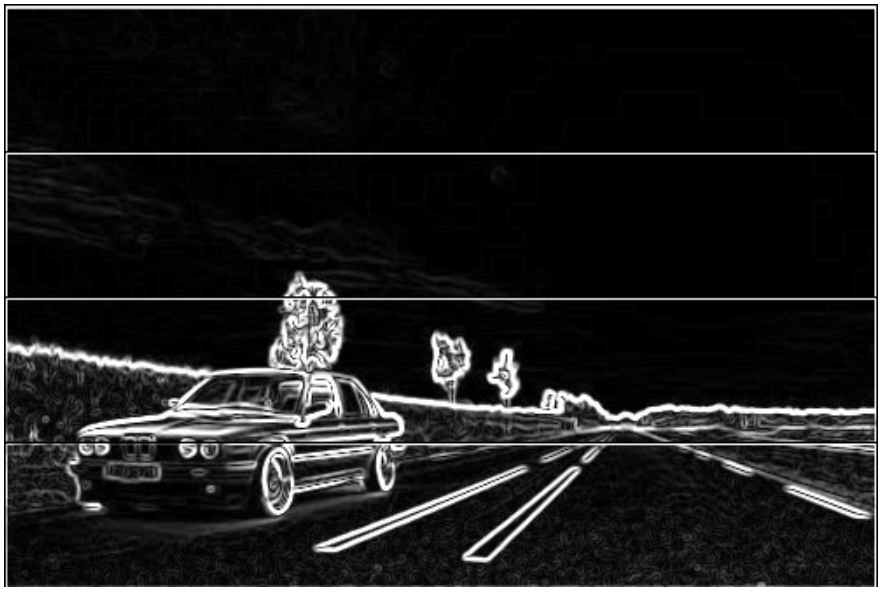


Fig. 12: MPI output showing horizontal partition-boundary artifacts when executed with 4 processes

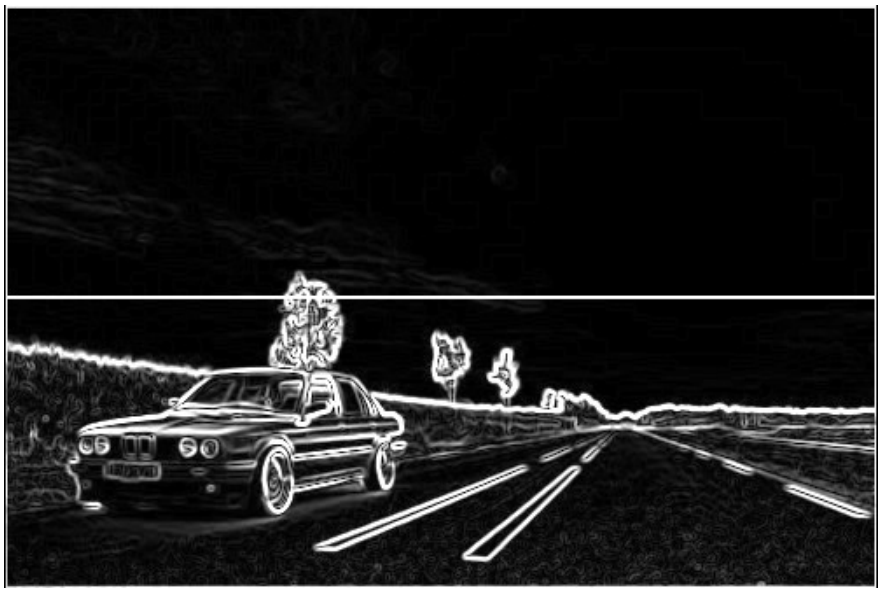


Fig. 13: MPI output showing boundary artifacts when executed with 2 processes prior to halo exchange correction

The issue was caused by incorrect handling of halo regions required for stencil-based filters such as Gaussian blur and Sobel edge detection. Initially, only grayscale halo rows were exchanged between neighboring processes, but the Sobel operator also depends on correctly blurred boundary pixels. As a result, missing boundary information produced visible discontinuities across partition edges.

To resolve this issue, a second halo exchange stage was introduced after the Gaussian blur operation. Additional halo buffers were allocated using `calloc()` to ensure proper initialization and prevent uninitialized memory artifacts. The corrected implementation exchanged blurred boundary rows before applying Sobel filtering, ensuring consistent convolution results across partition boundaries and eliminat-

ing visible stitching artifacts in the final image.The corrected halo exchange implementation is shown in Figure: 14

```
77  +   memcpy(gray_halos + w, local_gray, local_pixels);
83  78  int up = (rank == 0) ? MPI_PROC_NULL : rank - 1;
84  79  int down = (rank == size - 1) ? MPI_PROC_NULL : rank + 1;
85  80
86      -   // Send my top row to 'up', receive into my bottom halo from 'down'
87  81  MPI_Sendrecv(local_gray, w, MPI_UNSIGNED_CHAR, up, 0,
88      +               local_gray_halos + (local_rows + 1) * w, w, MPI_UNSIGNED_CHAR, down, 0,
    82  +               gray_halos + (local_rows + 1) * w, w, MPI_UNSIGNED_CHAR, down, 0,
89  83                  MPI_COMM_WORLD, MPI_STATUS_IGNORE);
90      -
91      -   // Send my bottom row to 'down', receive into my top halo from 'up'
92  84  MPI_Sendrecv(local_gray + (local_rows - 1) * w, w, MPI_UNSIGNED_CHAR, down, 1,
93      -               local_gray_halos, w, MPI_UNSIGNED_CHAR, up, 1,
    85  +               gray_halos, w, MPI_UNSIGNED_CHAR, up, 1,
94  86                  MPI_COMM_WORLD, MPI_STATUS_IGNORE);
95  87
96      -
    88  +   // 3. Blur
    89  +   blur_filter(gray_halos, blur_halos, w, local_rows + 2, false);
    90  +
    91  +   // 4. SECOND HALO EXCHANGE (This makes the blurred edges valid for Sobel)
    92  +   // We create a temp buffer of the blurred pixels to swap them
    93  +   unsigned char* temp_blur = malloc(local_pixels);
    94  +   for(int i = 0; i < local_rows; i++) {
```

Fig. 14: MPI halo exchange implementation used to correct boundary artifacts

After applying this fix, the horizontal artifact lines disappeared and the MPI output matched the serial baseline results across all processor configurations(Figure 15).



Fig. 15: Final MPI output after applying the corrected halo exchange mechanism.

VIII. Conclusion

This study evaluated the performance of OpenMP and MPI implementations for parallel image-processing operations, including grayscale conversion, Gaussian blur, and Sobel edge detection. Results show that both approaches improve performance compared to the serial baseline, with effectiveness depending on workload size and processor count.

OpenMP achieved better performance for moderate workloads due to efficient shared-memory parallelism and low communication overhead, while MPI showed stronger scalability for larger datasets where computation dominated communication cost. However, performance gains decreased at higher processor counts because of synchronization overhead, memory contention, and message-passing costs.

Overall, the results demonstrate that parallel performance is limited by communication overhead and serial components, highlighting the importance of selecting an appropriate parallelization strategy based on problem size and system architecture.

APPENDIX

Performance Formulas and Execution Outputs

A. Performance Metrics

$$\text{Speedup} = \frac{T_1}{T_p}$$

$$\text{Efficiency} = \frac{\text{Speedup}}{p}$$

where T_1 is the serial runtime, T_p is the runtime using p threads/processes, and p is the number of parallel workers.

B. Serial Execution Outputs

Figures 16–18 show terminal execution outputs for the serial implementation across different dataset sizes.

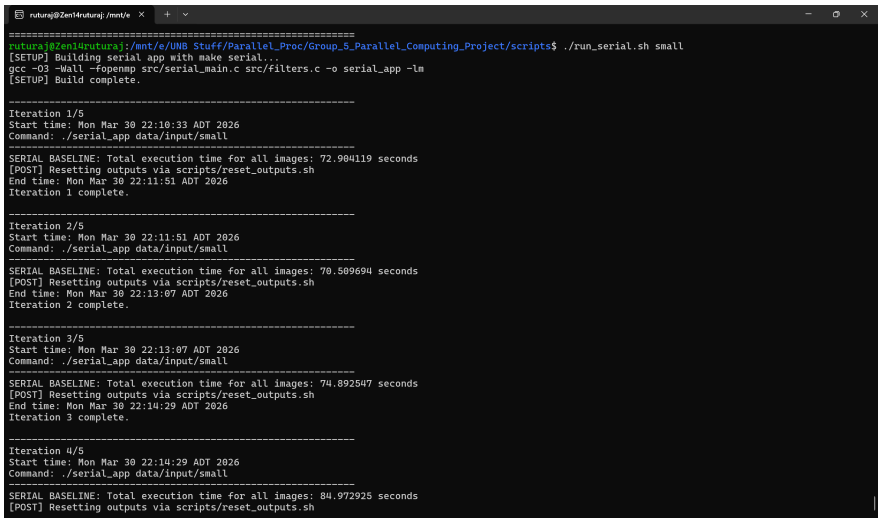


Fig. 16: Serial execution output for the small dataset

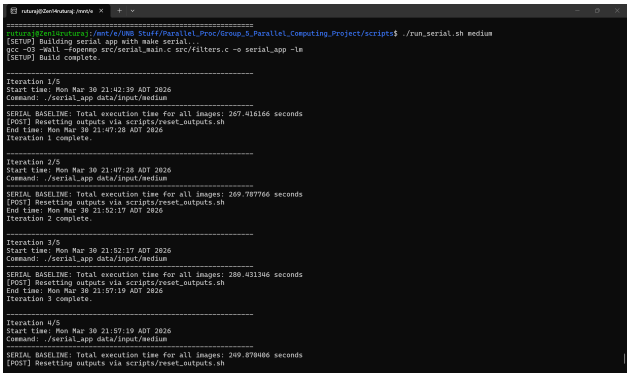


Fig. 17: Serial execution output for the medium dataset

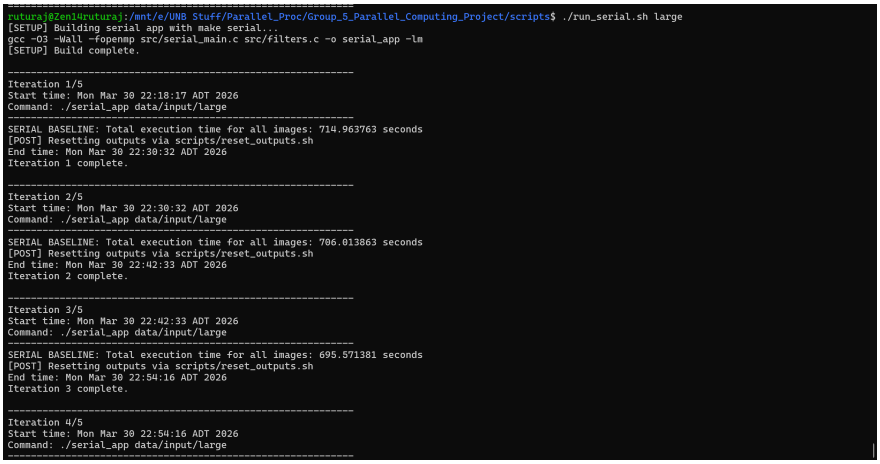


Fig. 18: Serial execution output for the large dataset

C. OpenMP Execution Outputs

Figures 21–27 show execution outputs for the OpenMP implementation using multiple thread configurations.

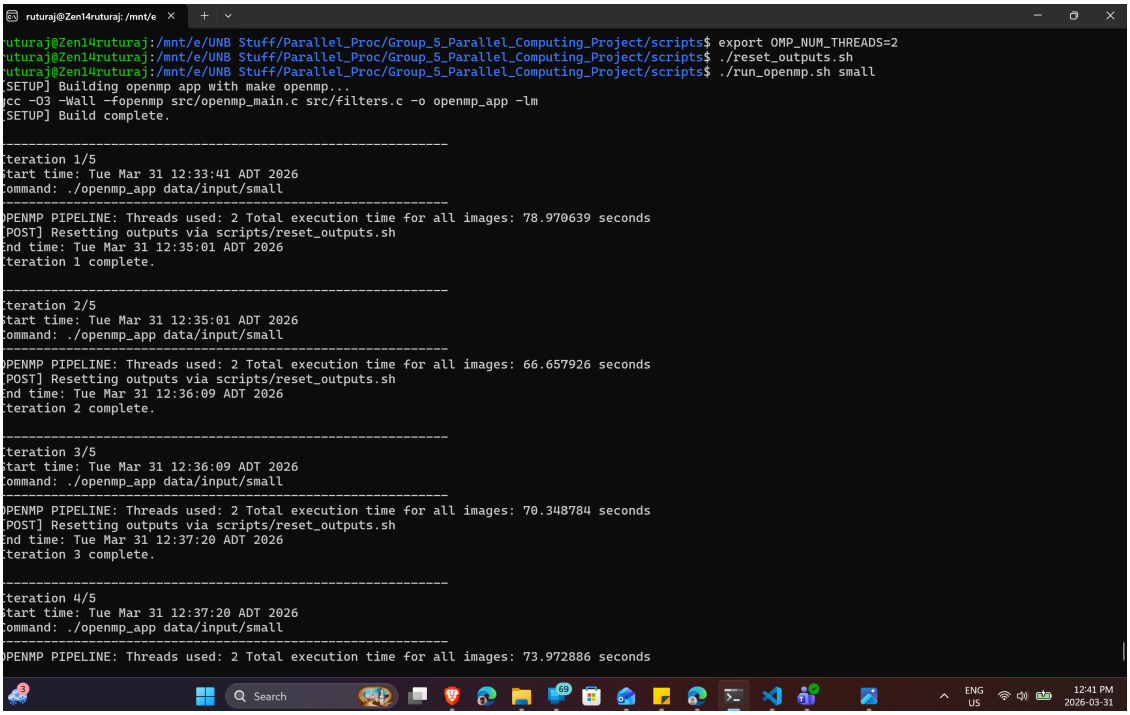


Fig. 19: OpenMP execution output for the small dataset with 2 threads

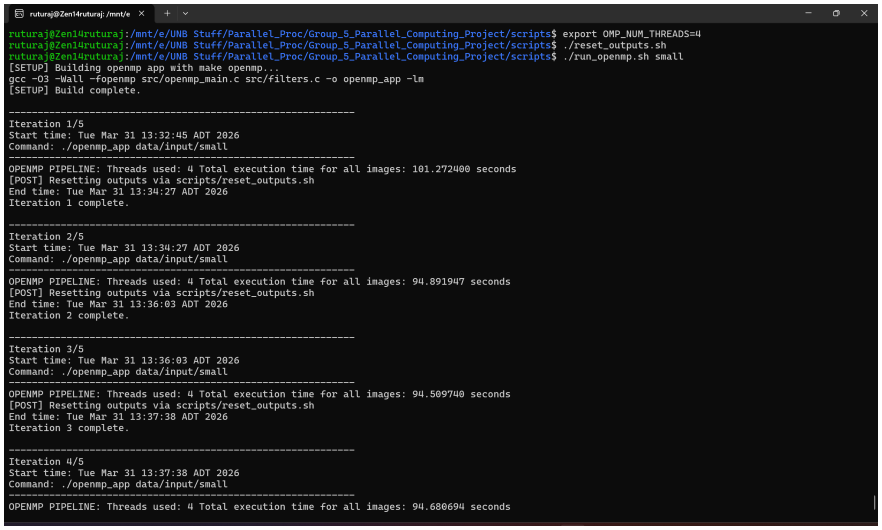


Fig. 20: OpenMP execution output for the small dataset with 4 threads

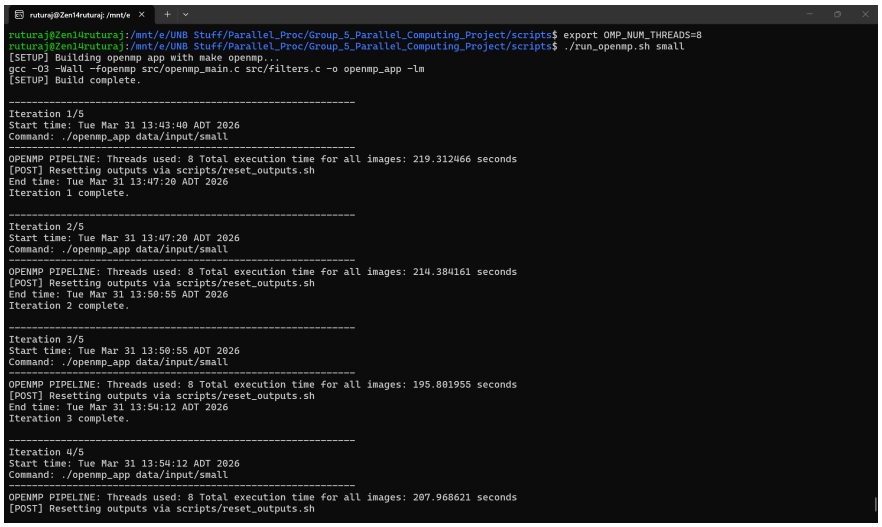


Fig. 21: OpenMP execution output for the small dataset with 8 threads

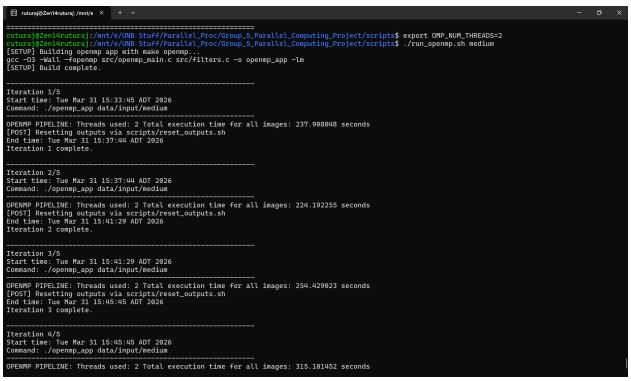


Fig. 22: OpenMP execution output for the medium dataset with 2 threads

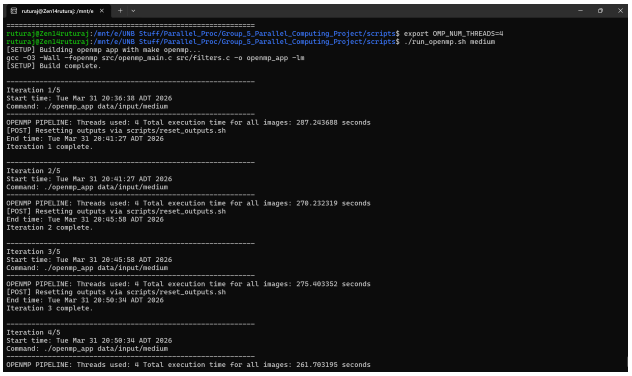


Fig. 23: OpenMP execution output for the medium dataset with 4 threads

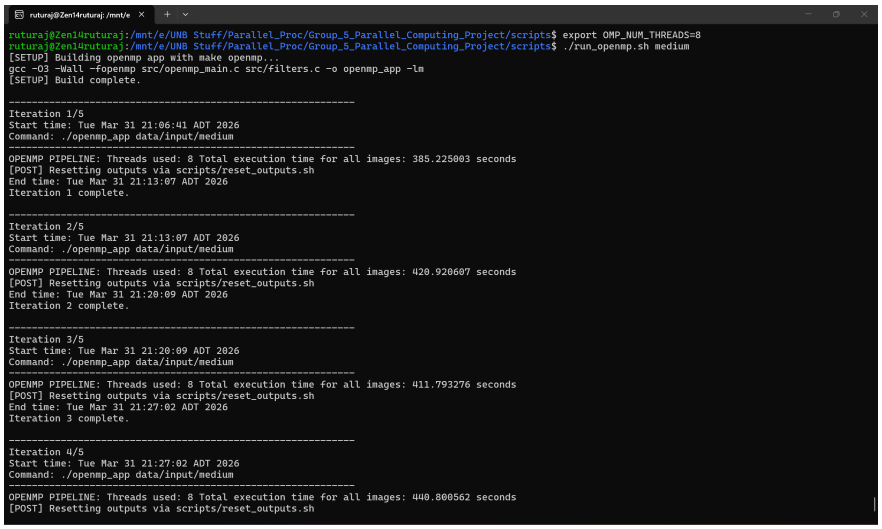


Fig. 24: OpenMP execution output for the medium dataset with 8 threads

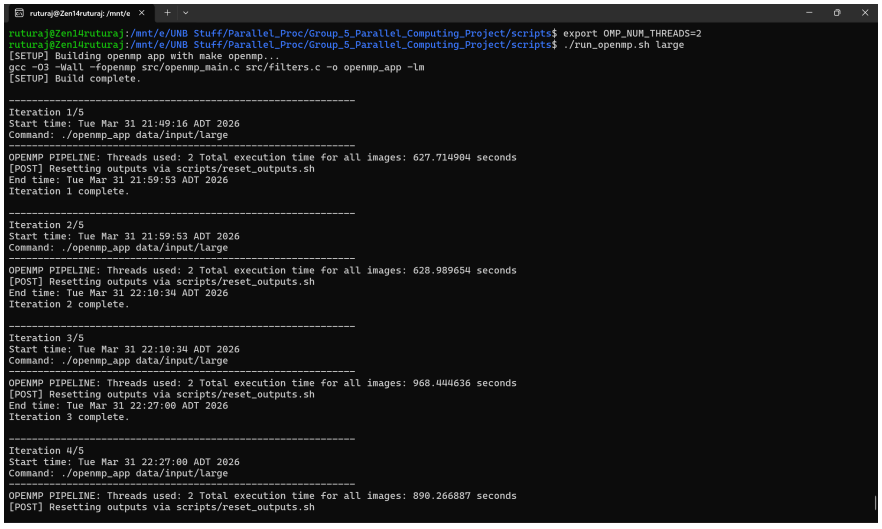


Fig. 25: OpenMP execution output for the large dataset with 2 threads

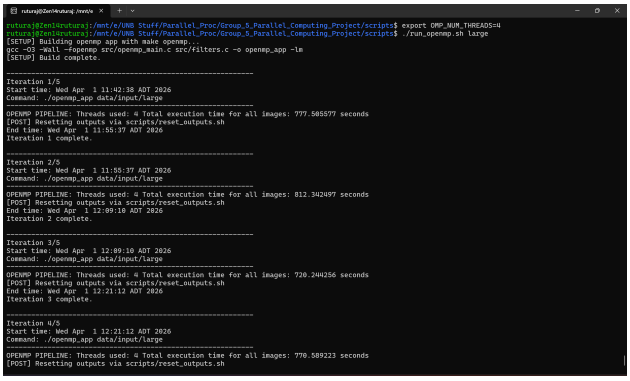


Fig. 26: OpenMP execution output for the large dataset with 4 threads

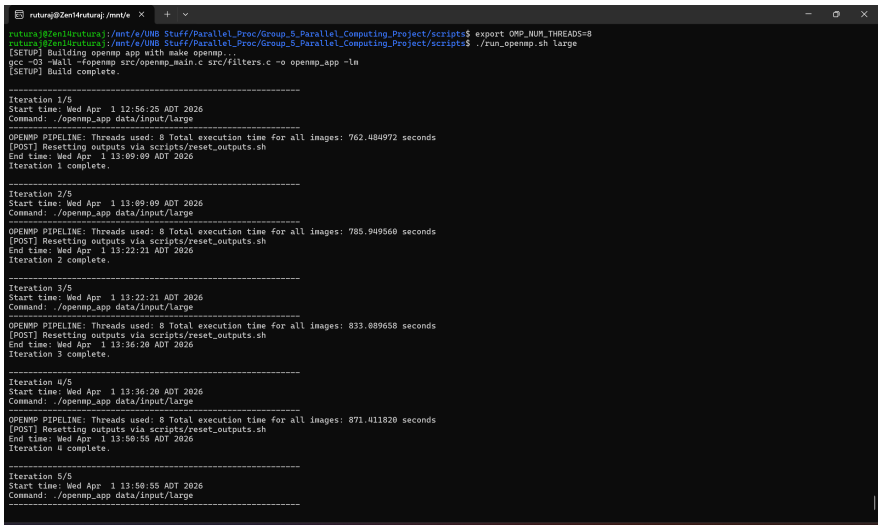


Fig. 27: OpenMP execution output for the large dataset with 8 threads

D. MPI Execution Outputs

Figures 30–36 show execution outputs for the MPI implementation using multiple process configurations.

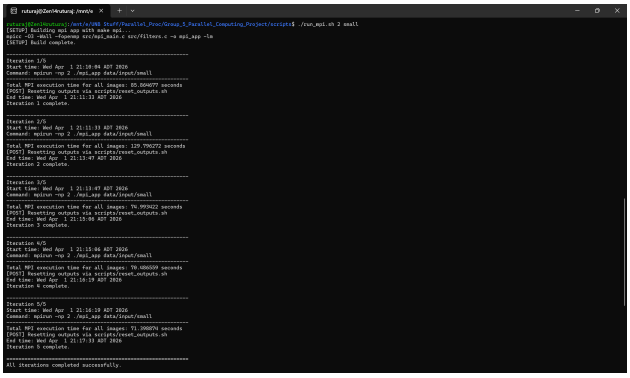


Fig. 28: MPI execution output for the small dataset with 2 processes

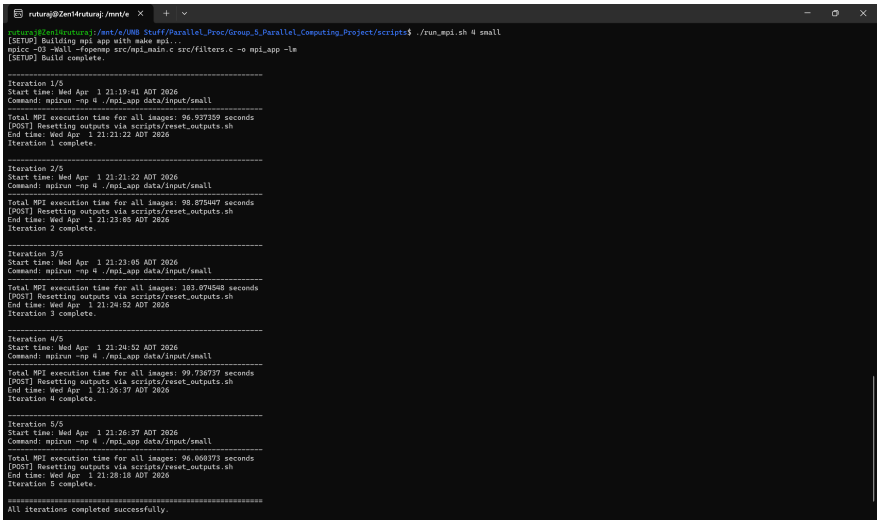


Fig. 29: MPI execution output for the small dataset with 4 processes

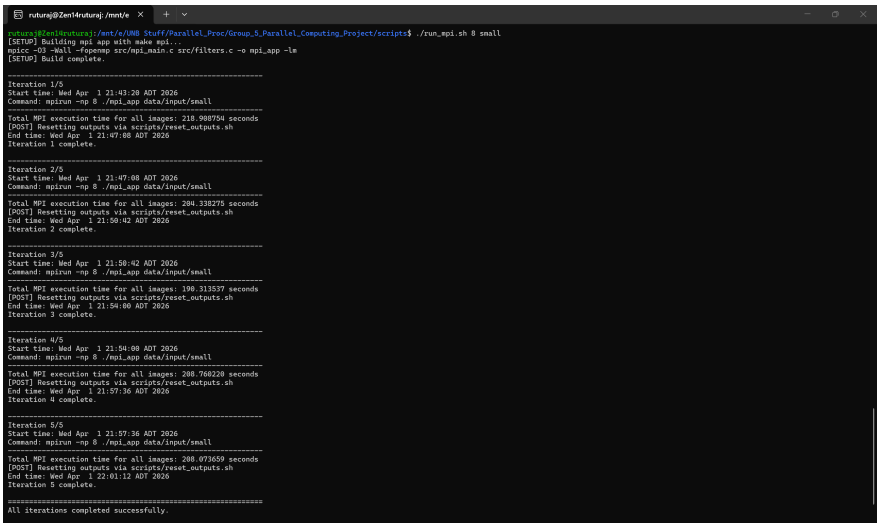


Fig. 30: MPI execution output for the small dataset with 8 processes

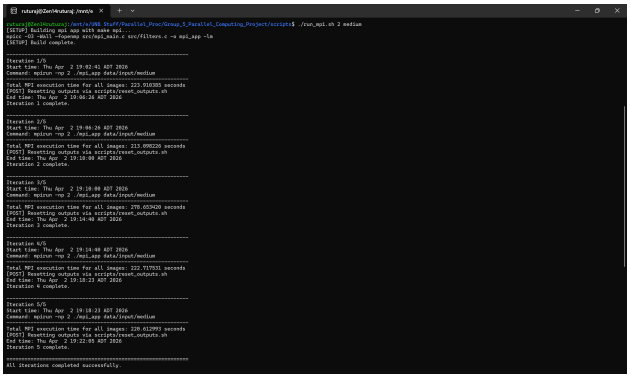


Fig. 31: MPI execution output for the medium dataset with 2 processes

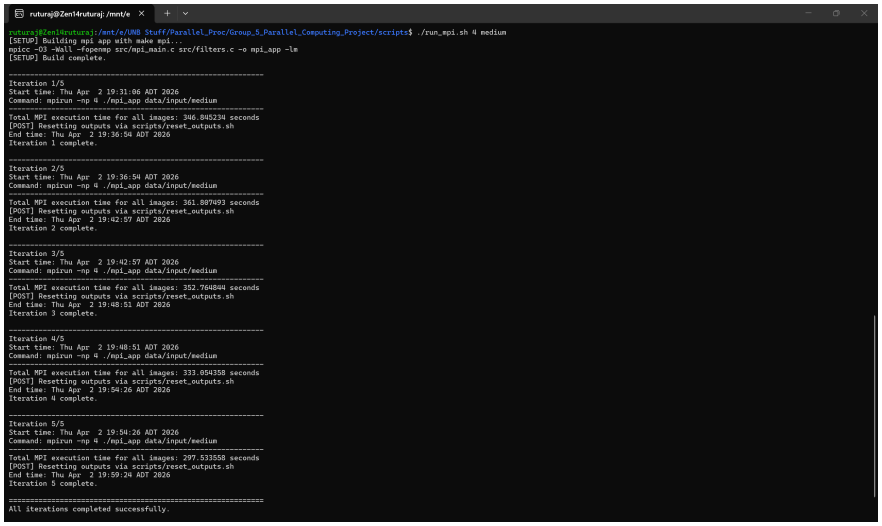


Fig. 32: MPI execution output for the medium dataset with 4 processes

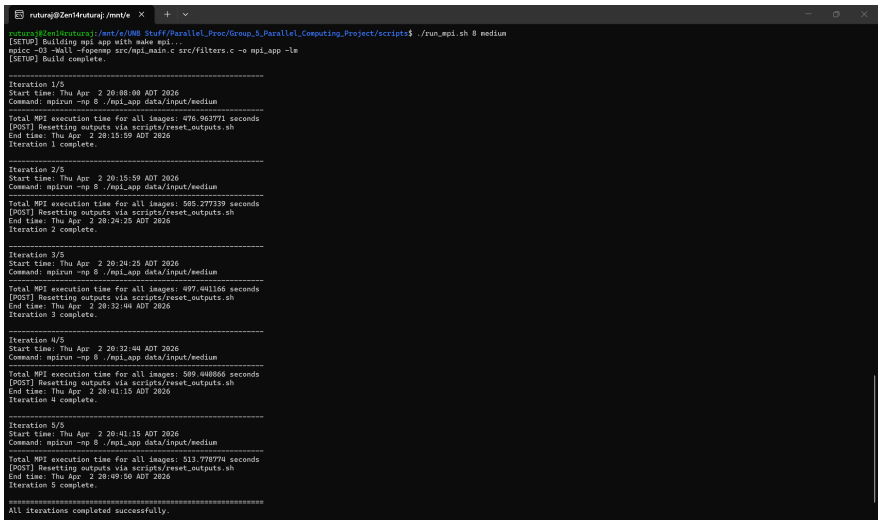


Fig. 33: MPI execution output for the medium dataset with 8 processes

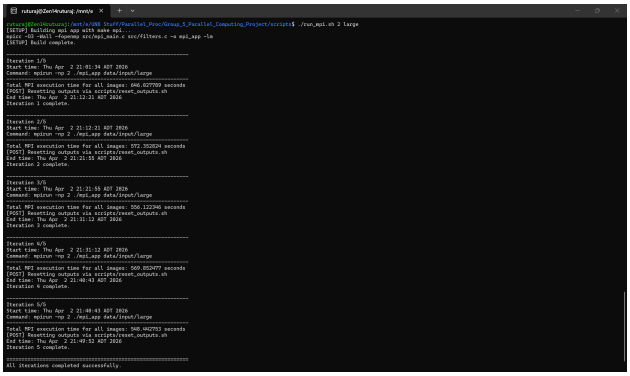


Fig. 34: MPI execution output for the large dataset with 2 processes

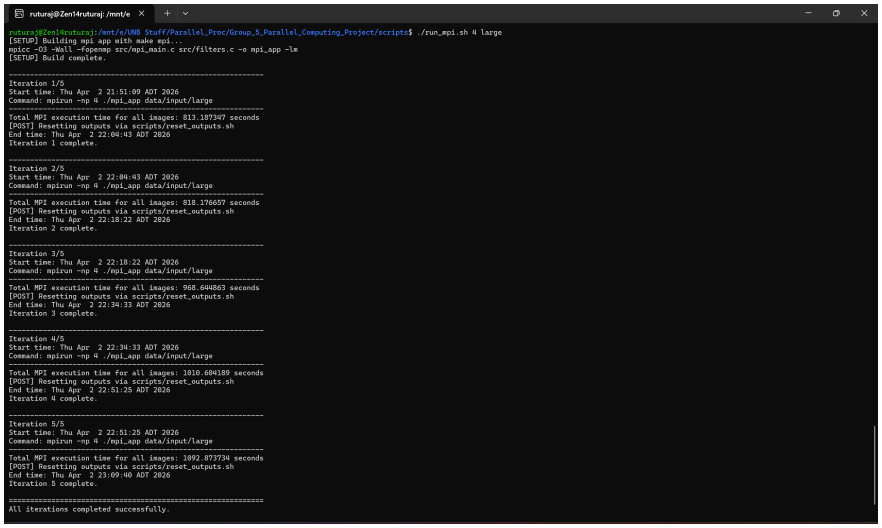


Fig. 35: MPI execution output for the large dataset with 4 processes

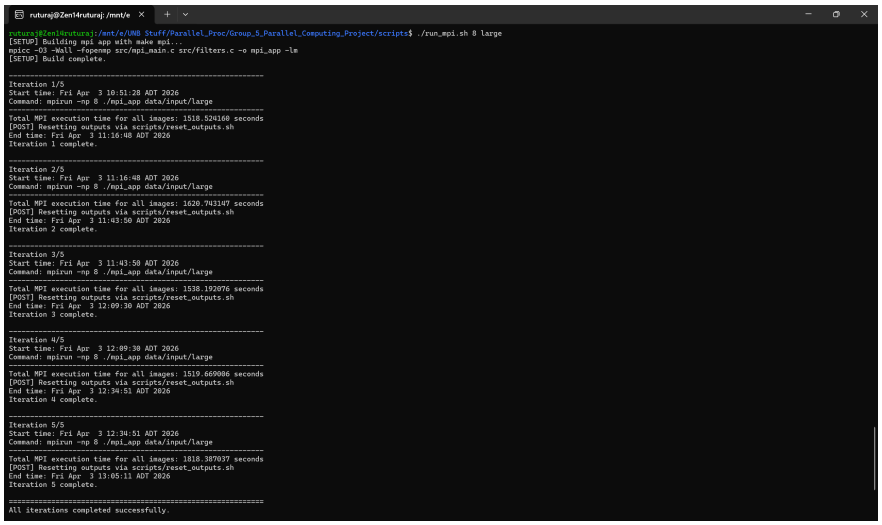


Fig. 36: MPI execution output for the large dataset with 8 processes

REFERENCES

[1] Mdfahimbinamin, "100k vehicle dashcam image dataset," https://www.kaggle.com/datasets/mdfahimbinamin/100k-vehicle-dashcam-image-dataset, 2023, accessed: 2026-03-23.

[2] S. Barrett, "stb: Single-file public domain libraries for c/c++," https://github.com/nothings/stb, 2017, accessed: 2026-03-31.

[3] J. J. Purushothaman, R. S. Ghodke, and T. M. Valliappan, "Autonomous vehicle vision system: Parallel computing performance analysis (openmp and mpi)," https://github.com/JayashreeJanani/Group_5_Parallel_Computing_Project.git, 2026, accessed: 2026-03-31.